

Universidad Tecnológica Nacional
Tecnicatura Universitaria en Programación a Distancia

Programación 1

Trabajo Práctico Integrador

Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Integrantes grupo n° 404:

Geronimo Jose Cardoso

Franco Ezequiel Ventrice

06 de junio de 2026

Índice

Marco Teórico	1
1. Listas	1
2. Diccionarios	1
3. Funciones	2
4. Condicionales	3
5. Ordenamientos	3
6. Estadísticas básicas	4
7. Archivos CSV	5
Decisiones Técnicas y Arquitectura	6
1. Diagrama de flujo de las operaciones principales	6
2. Capturas de pantalla de la ejecución	8
Dificultades y Conclusiones	12
1. Dificultades encontradas	12
2. Conclusiones	12
Links al repositorio y al video	13
Bibliografía	13

Marco Teórico

1. Listas

Una lista en Python es una estructura de datos **ordenada y mutable** que permite almacenar una colección de elementos bajo un mismo nombre. A diferencia de las variables simples que guardan un único valor, las listas pueden contener múltiples elementos como números, cadenas, booleanos o incluso otros diccionarios. Accesibles mediante un índice numérico que comienza en cero.

Sus características principales son:

- **Ordenadas:** los elementos mantienen el orden en que fueron insertados.
- **Mutables:** se pueden agregar, eliminar o modificar elementos después de su creación.
- **Indexadas:** cada elemento tiene una posición accesible con `lista[i]`.
- **Dinámicas:** no requieren un tamaño fijo, crecen o se reducen según sea necesario.

Las operaciones más comunes incluyen `.append()` para agregar un elemento al final, `len()` para obtener la cantidad de elementos, y `list()` para crear una copia de una lista existente sin modificar la original.

Aplicación en el proyecto: la variable `países` es la estructura central del sistema. Es una lista donde cada elemento es un diccionario que representa un país con sus cuatro atributos (nombre, población, superficie y continente). La función `cargar_paises()` construye esta lista leyendo el archivo CSV; `agregar_pais()` incorpora nuevos registros con `.append()`; `mostrar_lista_paises()` la recorre con un `for` para mostrar cada elemento; y `_ordenar_por_campo()` trabaja sobre una copia creada con `list(paises)` para no alterar la lista original durante el ordenamiento.

2. Diccionarios

Un diccionario en Python es una estructura de datos que almacena información en pares clave-valor, donde cada clave es única dentro del diccionario. Se definen con llaves `{}` y permiten acceder a cualquier valor directamente indicando su clave, sin necesidad de recorrer toda la estructura.

Sus características principales son:

- **No ordenados por inserción:** los diccionarios mantienen el orden de inserción.
- **Claves únicas:** no se pueden repetir. Si se asigna un valor a una clave existente, el valor anterior se sobrescribe.
- **Mutables:** sus valores pueden modificarse después de la creación.
- **Acceso directo:** se accede a un valor mediante `diccionario["clave"]`, sin necesidad de índices numéricos.

Aplicación en el proyecto: cada país del dataset está representado como un diccionario con cuatro claves fijas: `nombre`, `poblacion`, `superficie` y `continente`. Esta estructura permite acceder a cualquier atributo del país de forma clara y legible. En `busqueda.py`, se consulta `pais["nombre"]` para comparar con el término buscado; en `estadisticas.py`, se accede a `pais["poblacion"]` y `pais["superficie"]` para calcular máximos, mínimos y promedios; y en `utils.py`, se recorren todas las claves para mostrar la información formateada al usuario.

3. Funciones

Una función es un bloque de código reutilizable que encapsula una tarea específica y puede ser invocado múltiples veces desde distintos puntos del programa. En Python se definen con la palabra clave `def`, seguida del nombre y sus parámetros entre paréntesis. Pueden recibir datos a través de parámetros, operar sobre ellos y devolver un resultado mediante `return`.

Un principio fundamental en el diseño de funciones es la **modularización**: cada función debe tener una única responsabilidad. Este principio mejora la legibilidad del código, facilita el mantenimiento y reduce la duplicación. Cuando una tarea requiere ser auxiliada por otra función de uso interno, Python adopta la convención de nombrarla con un guión bajo al inicio (`_nombre_funcion`), indicando que es privada y no debe ser llamada directamente desde fuera del módulo.

Los parámetros pueden tener **valores por defecto**, lo que permite que sean opcionales al momento de llamar la función. Por ejemplo, `def cargar_paises(ruta=ARCHIVO_CSV)` permite invocar la función sin argumentos y usar el archivo predeterminado, o especificar una ruta diferente cuando sea necesario.

Aplicación en el proyecto: el sistema completo está organizado en funciones distribuidas en seis módulos, donde cada función cumple exactamente una responsabilidad.

`cargar_paises()` se encarga solo de leer; `guardar_paises()` solo de escribir; `agregar_pais()` solo de incorporar nuevos registros. La función auxiliar `_ordenar_por_campo()` usa el guion bajo porque es de uso interno de

`ordenar_paises()`. Este diseño permite que `main.py` orqueste el flujo general del programa sin conocer los detalles de implementación de cada operación.

4. Condicionales

Las estructuras condicionales permiten definir distintos caminos de ejecución dentro de un programa en función de si una o más condiciones se cumplen o no. En Python se implementan con las palabras clave `if`, `elif` y `else`. El bloque `if` se ejecuta cuando la condición es verdadera; `elif` permite evaluar condiciones adicionales; y `else` define el camino alternativo cuando ninguna condición anterior se cumplió.

Sus características principales son:

- **Evaluación booleana:** la condición se evalúa como `True` o `False`.
- **Encadenamiento:** se pueden combinar múltiples condiciones con `elif`.
- **Cortocircuito con `return`:** dentro de una función, un `if` seguido de `return` interrumpe la ejecución inmediatamente, evitando continuar con código innecesario.
- **Comparaciones encadenadas:** Python permite expresiones como `minimo <= valor <= maximo` como una única condición, lo cual nos da un código más limpio.

Aplicación en el proyecto: los condicionales se utilizan principalmente para dos propósitos. El primero es la validación de entradas del usuario: por ejemplo, `if not continente` en `filtrar_por_continente()` verifica que el campo no quede vacío antes de continuar, y `if minimo > maximo` en `filtrar_por_poblacion()` detecta rangos inválidos. El segundo es la lógica de comparación en las funciones auxiliares de `estadisticas.py`: `_pais_mayor()` y `_pais_menor()` usan `if pais[campo] > mayor[campo]` e `if pais[campo] < menor[campo]` respectivamente para recorrer la lista y determinar el país con el valor máximo o mínimo en el campo indicado.

5. Ordenamientos

Los algoritmos de ordenamiento son procedimientos que reorganizan los elementos de una colección según un criterio definido, generalmente de menor a mayor (ascendente) o de mayor a menor (descendente). El ordenamiento es una operación fundamental en el desarrollo de software: facilita la búsqueda eficiente de datos, mejora la presentación de información al usuario y es la base de numerosos algoritmos más avanzados.

Existen múltiples algoritmos de ordenamiento, cada uno con distintas características de eficiencia. Entre los más conocidos se encuentran el **Bubble Sort**, **Selection Sort**,

Insertion Sort, Merge Sort y Quick Sort. En este proyecto se implementó el **Bubble Sort mejorado**, que opera comparando pares de elementos adyacentes e intercambiándolos si no están en el orden correcto. La versión "mejorada" incorpora una optimización: si durante una pasada completa no se realizó ningún intercambio, la lista ya está ordenada y el algoritmo se detiene anticipadamente mediante la instrucción **break**, evitando iteraciones innecesarias.

El proceso del Bubble Sort sobre la lista `[5, 2, 4, 1]` sería:

1. Primera pasada: el mayor elemento "burbujea" al final → `[2, 4, 1, 5]`
2. Segunda pasada: el segundo mayor queda en su lugar → `[2, 1, 4, 5]`
3. Tercera pasada: → `[1, 2, 4, 5]`
4. Cuarta pasada: sin intercambios → lista ordenada, el **break** detiene el proceso.

Aplicación en el proyecto: la función `_ordenar_por_campo(países, campo, ascendente)` implementa el Bubble Sort adaptado para listas de diccionarios. En lugar de comparar elementos directamente (`lista[j] > lista[j+1]`), compara el valor de un campo específico del diccionario: `copia[j][campo] > copia[j+1][campo]`. El parámetro `ascendente` controla la dirección: `True` usa el operador `>` (los mayores van al final), `False` usa `<` (los menores van al final). La función opera siempre sobre una copia de la lista original para garantizar que el dataset no se modifique.

6. Estadísticas básicas

Las estadísticas básicas son operaciones matemáticas simples que permiten resumir y describir un conjunto de datos, facilitando la interpretación de la información por parte del usuario. En el contexto del desarrollo de software, estas operaciones se aplican sobre colecciones de datos para obtener indicadores clave sin necesidad de analizar cada registro individualmente.

Las operaciones utilizadas en este proyecto son:

- **Máximo y mínimo:** identifican el elemento con el valor más alto o más bajo en un campo determinado, recorriendo toda la colección y comparando elemento a elemento.
- **Promedio:** se obtiene sumando todos los valores del campo y dividiendo el resultado por la cantidad de elementos. Devuelve un valor `float` que representa el valor central del conjunto.

- **Conteo agrupado:** cuenta cuántos elementos pertenecen a cada categoría, acumulando los resultados en un diccionario donde cada clave es una categoría y su valor es la cantidad correspondiente.

Aplicación en el proyecto: estas operaciones se implementaron en `estadisticas.py` a través de cuatro funciones auxiliares privadas. `_pais_mayor()` y `_pais_menor()` reciben la lista de países y el nombre del campo ("`poblacion`" o "`superficie`") y retornan el diccionario completo del país con el valor máximo o mínimo respectivamente. `_promedio()` acumula la suma del campo indicado y divide por `len(paises)`, retornando el resultado como `float`. `_cantidad_por_continente()` recorre la lista y construye un diccionario acumulador que agrupa y cuenta los países por continente, retornándolo ordenado alfabéticamente. Las cuatro son coordinadas por `mostrar_estadisticas()`, que las llama y muestra los resultados formateados al usuario.

7. Archivos CSV

Un archivo CSV (*Comma-Separated Values*, Valores Separados por Comas) es un formato de texto plano utilizado para almacenar datos en forma de tabla. Cada línea del archivo representa una fila de la tabla, y los valores de cada columna se separan por un delimitador —generalmente una coma, aunque en países de habla hispana también se usa el punto y coma para evitar conflictos con el separador decimal. La primera línea del archivo suele contener los **encabezados**, que definen el nombre de cada columna.

Este formato es ampliamente utilizado en el desarrollo de software por varias razones: su **interoperabilidad** (puede ser procesado por cualquier lenguaje de programación), su **ligereza** (texto plano sin formato propietario), su **persistencia** (permite guardar datos de la RAM al disco de manera permanente) y su facilidad de edición manual. Programas como Microsoft Excel y Google Sheets pueden abrir y editar archivos CSV directamente.

Python incluye el módulo `csv` en su biblioteca estándar, con herramientas especializadas para su manejo:

- **`csv.DictReader`:** lee cada fila y la convierte automáticamente en un diccionario, usando los encabezados como claves. Elimina la necesidad de gestionar índices de columna y se adapta automáticamente si el orden de las columnas cambia.
- **`csv.DictWriter`:** escribe listas de diccionarios en el archivo respetando el orden definido en el parámetro `fieldnames`, y genera el encabezado automáticamente con `writeheader()`.

Es fundamental usar `encoding="utf-8"` para soportar caracteres especiales como tildes y ñ, y `newline=""` al escribir en Windows para evitar filas vacías adicionales entre registros.

Aplicación en el proyecto: el archivo `países.csv` es la capa de persistencia del sistema. `cargar_países()` usa `DictReader` para convertir cada fila en un diccionario y aplica `int()` para convertir `población` y `superficie` de texto a número entero, ya que el CSV almacena todos los valores como cadenas. `guardar_países()` usa `DictWriter` con la constante `CAMPOS = ["nombre", "poblacion", "superficie", "continente"]` como `fieldnames`, garantizando el orden de columnas. El manejo de errores cubre `FileNotFoundError` (archivo no encontrado) y `ValueError` (dato numérico mal formado), devolviendo una lista vacía en ambos casos para no interrumpir el programa.

Decisiones Técnicas y Arquitectura

1. Diagrama de flujo de las operaciones principales

El siguiente diagrama representa el flujo principal de ejecución del programa. Al iniciarse, el sistema carga el dataset desde el archivo `países.csv`. Si la carga es exitosa, se presenta al usuario un menú de opciones en bucle continuo. Las operaciones que modifican datos (agregar y actualizar) escriben los cambios en el CSV antes de volver al menú. Las operaciones de consulta (búsqueda, filtros, ordenamiento, estadísticas y visualización) muestran el resultado en pantalla sin alterar el archivo. El programa finaliza cuando el usuario elige la opción 0 (Salir).

Diagrama de flujo — Gestión de Datos de Países

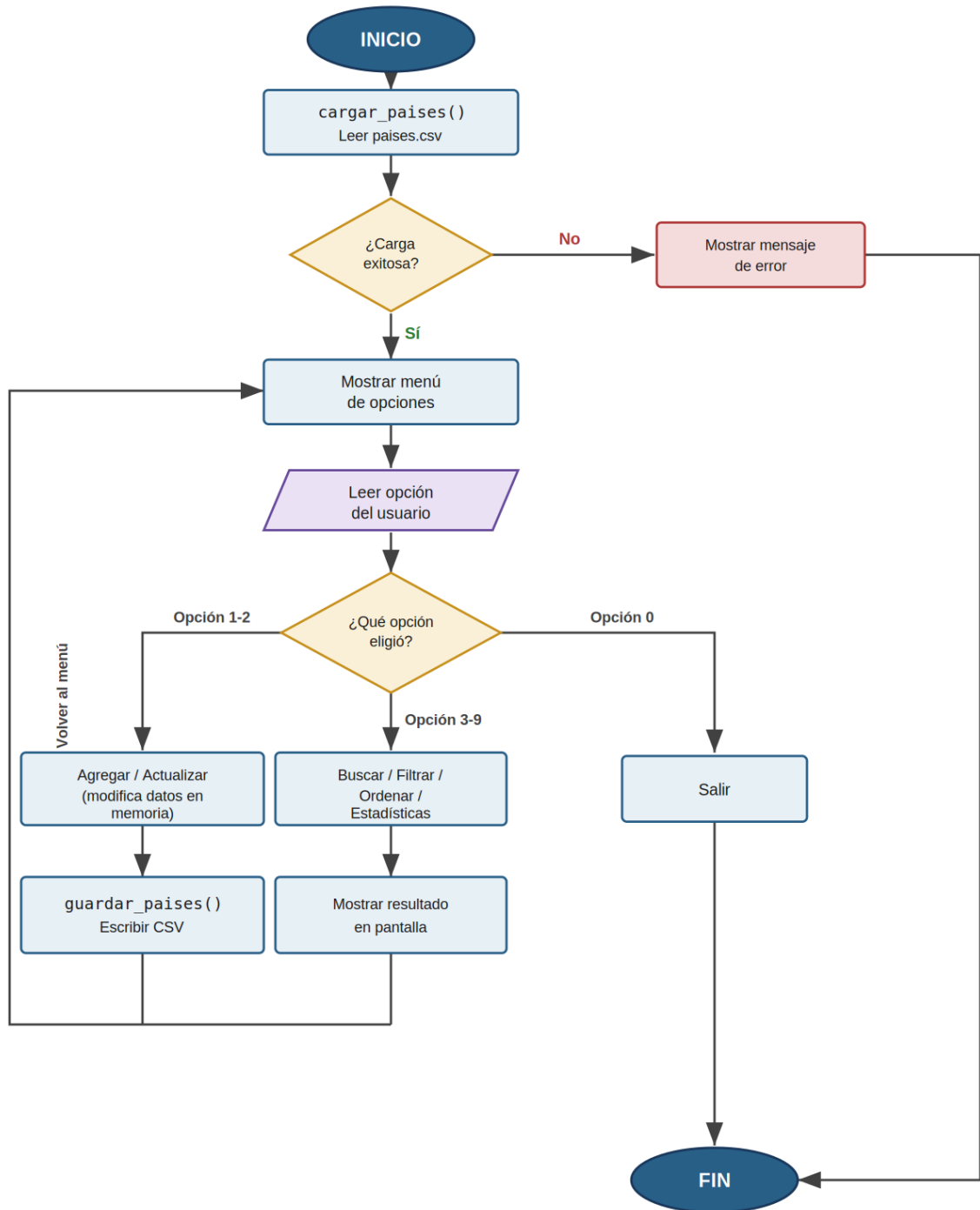
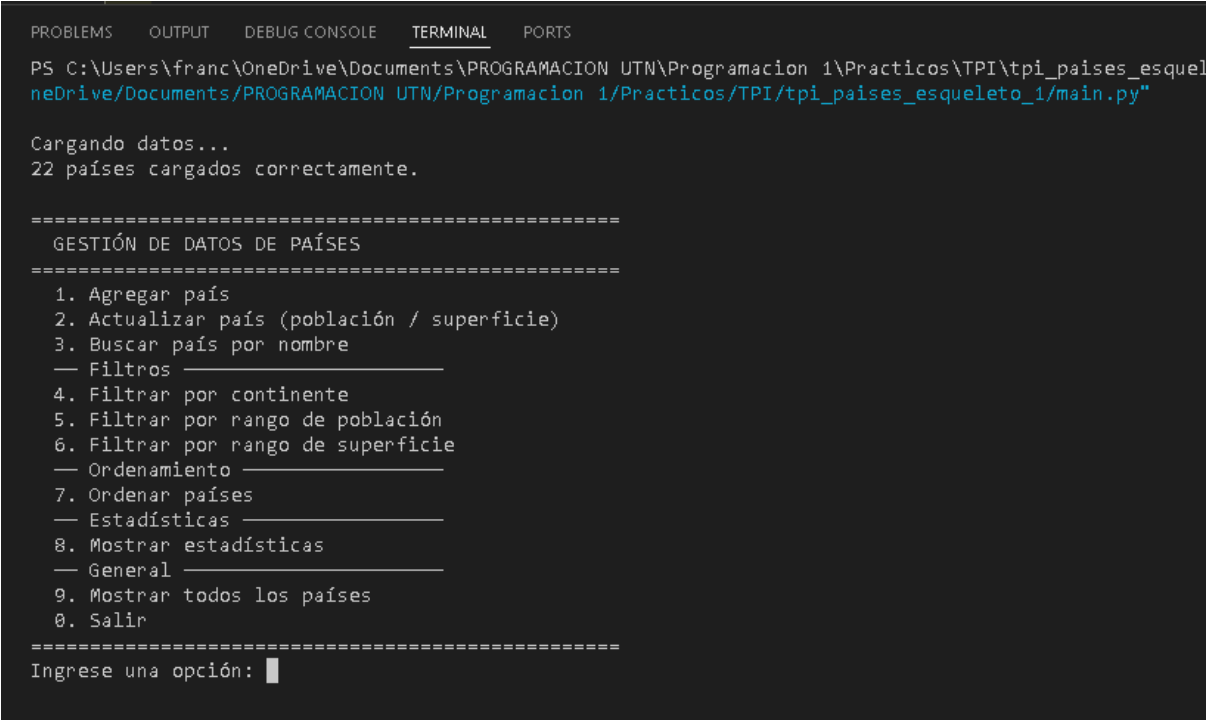


Figura 1: Diagrama de flujo del sistema de gestión de países. Elaboración propia.

2. Capturas de pantalla de la ejecución

A continuación se presentan capturas de pantalla del programa en ejecución, ilustrando las principales funcionalidades implementadas.



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\franc\OneDrive\Documents\PROGRAMACION UTN\Programacion 1\Practicos\TPI\tpi_paises_esquel
neDrive/Documents/PROGRAMACION UTN/Programacion 1/Practicos/TPI/tpi_paises_esqueleto_1/main.py"

Cargando datos...
22 países cargados correctamente.

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Agregar país
2. Actualizar país (población / superficie)
3. Buscar país por nombre
  — Filtros —————
4. Filtrar por continente
5. Filtrar por rango de población
6. Filtrar por rango de superficie
  — Ordenamiento —————
7. Ordenar países
  — Estadísticas —————
8. Mostrar estadísticas
  — General —————
9. Mostrar todos los países
0. Salir
=====
Ingrese una opción: █
```

Figura 2: Captura del menú principal mostrando que el programa inicia y carga el csv

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\franc\OneDrive\Documents\PROGRAMACION UTN\Programacion 1\Pro
neDrive/Documents/PROGRAMACION UTN/Programacion 1/Practicos/TPI/tpi_pais

Cargando datos...
22 países cargados correctamente.

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Agregar país
2. Actualizar país (población / superficie)
3. Buscar país por nombre
  — Filtros —————
4. Filtrar por continente
5. Filtrar por rango de población
6. Filtrar por rango de superficie
  — Ordenamiento —————
7. Ordenar países
  — Estadísticas —————
8. Mostrar estadísticas
  — General —————
9. Mostrar todos los países
0. Salir
=====
Ingrese una opción: 1
Ingrese el nombre del país: Holanda
Ingrese la poblacion del país: 7273626272
Ingrese la superficie del país: 7463637
Ingrese el continente donde se encuentra ubicado el país: Europa

Presione Enter para continuar...
```

Figura 3: Captura de la opción 1 del menú “Agregar país”

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

5. Filtrar por rango de población
6. Filtrar por rango de superficie
— Ordenamiento —
7. Ordenar países
— Estadísticas —
8. Mostrar estadísticas
— General —
9. Mostrar todos los países
0. Salir

=====
Ingrese una opción: 7
Indique el criterio de ordenamiento:
1. Por Nombre
2. Por Poblacion
3. Por Superficie
2
Ingrese direccion:
A. Ascendente
D. Descendente
d
=====
Países ordenados
=====
Nombre:      Holanda
Continente:  Europa
Población:   7.273.626.272
Superficie:  7.463.637 km²
-----
Nombre:      China
Continente:  Asia
Población:   1.412.600.000
Superficie:  9.596.960 km²
-----
Nombre:      India
Continente:  Asia
Población:   1.380.004.385
Superficie:  3.287.263 km²

```

Figura 4: Captura de la opción 7 del menú “Ordenar países”. Donde se observa que se ordenaron por población de manera descendente.

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

4. Filtrar por continente
5. Filtrar por rango de población
6. Filtrar por rango de superficie
— Ordenamiento —
7. Ordenar países
— Estadísticas —
8. Mostrar estadísticas
— General —
9. Mostrar todos los países
0. Salir

=====
Ingrese una opción: 8

=====

ESTADÍSTICAS
=====

Total de países cargados: 23

- Mayor población:    Holanda (7.273.626.272)
- Menor población:    Uruguay (3.473.727)
- Promedio de población: 509.658.658

- Mayor superficie:    Canada (9.879.750)
- Menor superficie:    Portugal (92.212)
- Promedio de superficie: 2.366.553

Países por continente:
  América: 8
  Asia: 5
  Europa: 7
  África: 3

=====

Presione Enter para continuar...|
```

Figura 5: Captura de la opción 8 del menú “Mostrar estadísticas”.

Dificultades y Conclusiones

1. Dificultades encontradas

- 1. Adaptar el Bubble Sort para diccionarios** El algoritmo del apunte trabaja con listas de números simples. Adaptarlo para comparar un campo específico de cada diccionario (`copia[j][campo]`) requirió entender cómo acceder a los valores por clave y cómo parametrizar la comparación según el criterio elegido.
- 2. Manejo de tipos en el CSV** El módulo `csv` lee todos los valores como strings. Fue necesario convertir `poblacion` y `superficie` a `int` al cargar, y manejar el `ValueError` cuando un dato estaba mal formado.
- 3. Separación de responsabilidades entre módulos** Definir qué función escribe el CSV y cuál solo modifica la lista en memoria (el contrato `return True/False` entre `agregar_pais` y `guardar_paises`) requirió pensar la arquitectura antes de implementar.
- 4. Formato de números para el usuario** Python usa coma como separador de miles (`:`), pero la convención argentina usa punto. Fue necesario combinar el especificador de formato con `.replace(",", ".", ".")` para mostrar los números correctamente.

2. Conclusiones

El desarrollo de este trabajo práctico permitió aplicar en un contexto real los conceptos teóricos trabajados durante la cursada. La organización del código en módulos con responsabilidades bien definidas facilitó el trabajo colaborativo y redujo los errores de integración.

Comprendimos la importancia de la persistencia mediante archivos CSV como alternativa simple y portable a una base de datos, y la diferencia entre modificar datos en memoria y escribirlos en disco.

La implementación manual del Bubble Sort mejorado, en lugar de usar `sorted()`, nos permitió entender en profundidad el funcionamiento interno de los algoritmos de ordenamiento y sus condiciones de optimización.

Como aprendizaje general, el proyecto demostró que la modularización no es solo una buena práctica sino una necesidad real cuando múltiples personas trabajan sobre el mismo código.

Para la definición de la arquitectura del sistema, la división de tareas entre los integrantes y la revisión del código desarrollado, se utilizó Claude (Anthropic) como herramienta de

apoyo. El código fue escrito e implementado por los integrantes del equipo, comprendiendo y pudiendo defender cada decisión de diseño tomada

Links al repositorio y al video

Link al video explicativo: <https://youtu.be/V4IHhQ9NKIU>

Link al repositorio en GitHub: <https://github.com/ventricef012-ai/tpi-paises>

Bibliografía

1. **Python Software Foundation.** (2024). *Python 3 Documentation — Data Structures*. <https://docs.python.org/3/tutorial/datastructures.html>
2. **Python Software Foundation.** (2024). *Python 3 Documentation — csv module*. <https://docs.python.org/3/library/csv.html>
3. **Sweigart, A.** (2020). *Automate the Boring Stuff with Python* (2da ed.). No Starch Press. <https://automatetheboringstuff.com>
4. **Cátedra de Programación 1.** (2026). *Material didáctico: Listas, archivos CSV y algoritmos de ordenamiento*. UTN — Tecnicatura Universitaria en Programación a Distancia.
5. **Anthropic.** (2026). *Claude — AI Assistant*. <https://www.anthropic.com>